



Cyberscope

A *TAC Security* Company

Audit Report

UCBI Banking Token

April 2026

Network ETH

Address 0x2adba23Cf1252dE095aCEd801e758b369EC10426

Audited by © cyberscope

Analysis

● Critical ● Medium ● Minor / Informative ● Pass

Severity	Code	Description	Status
●	ST	Stops Transactions	Acknowledged
●	OTUT	Transfers User's Tokens	Passed
●	ELFM	Exceeds Fees Limit	Passed
●	MT	Mints Tokens	Passed
●	BT	Burns Tokens	Passed
●	BC	Blacklists Addresses	Acknowledged

Diagnostics

● Critical
 ● Medium
 ● Minor / Informative

Severity	Code	Description	Status
●	ITC	Insufficient Transfer Check	Acknowledged
●	CCR	Contract Centralization Risk	Acknowledged
●	IAU	Inefficient Assert Usage	Acknowledged
●	MC	Missing Check	Acknowledged
●	MEM	Missing Error Messages	Acknowledged
●	MEE	Missing Events Emission	Acknowledged
●	MRO	Missing Renounce Ownership	Acknowledged
●	MTEE	Missing Transfer Event Emission	Acknowledged
●	NDAB	Non-Reverting Decrease Approval Behavior	Acknowledged
●	NWES	Nonconformity with ERC-20 Standard	Acknowledged
●	OCV	Outdated Compiler Version	Acknowledged
●	L04	Conformance to Solidity Naming Conventions	Acknowledged
●	L09	Dead Code Elimination	Acknowledged
●	L19	Stable Compiler Version	Acknowledged

Table of Contents

Analysis	1
Diagnostics	2
Table of Contents	3
Risk Classification	5
Review	6
Audit Updates	6
Source Files	6
Overview	7
1. Introduction	7
2. The UCBI Platform	7
3. Role of the UCBI Token in the Ecosystem	8
4. Relationship to the Audited Contract	9
5. Scope of this Overview	9
Findings Breakdown	10
ST - Stops Transactions	11
Description	11
Recommendation	12
Team Update	13
BC - Blacklists Addresses	15
Description	15
Team Update	16
ITC - Insufficient Transfer Check	18
Description	18
Recommendation	18
CCR - Contract Centralization Risk	21
Description	21
Recommendation	21
Team Update	22
IAU - Inefficient Assert Usage	24
Description	24
Recommendation	24
Team Update	24
MC - Missing Check	25
Description	25
Recommendation	25
Team Update	26
MEM - Missing Error Messages	27
Description	27
Recommendation	27

Team Update	27
MEE - Missing Events Emission	28
Description	28
Recommendation	28
Team Update	29
MRO - Missing Renounce Ownership	30
Description	30
Recommendation	30
Team Update	30
MTEE - Missing Transfer Event Emission	31
Description	31
Recommendation	31
Team Update	31
NDAB - Non-Reverting Decrease Approval Behavior	32
Description	32
Recommendation	32
Team Update	33
NWES - Nonconformity with ERC-20 Standard	34
Description	34
Recommendation	34
Team Update	34
OCV - Outdated Compiler Version	35
Description	35
Recommendation	35
Team Update	35
L04 - Conformance to Solidity Naming Conventions	36
Description	36
Recommendation	37
Team Update	37
L09 - Dead Code Elimination	38
Description	38
Recommendation	38
Team Update	39
L19 - Stable Compiler Version	40
Description	40
Recommendation	40
Functions Analysis	41
Inheritance Graph	43
Flow Graph	44
Summary	45
Disclaimer	46
About Cyberscope	47

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Contract Name	UCBI Banking
Explorer	https://etherscan.io/address/0x2adba23cf1252de095aced801e758b369ec10426
Symbol	UCBI
Decimals	8
Total Supply	12,000,000

Audit Updates

Initial Audit	21 Apr 2026
----------------------	-------------

Source Files

Filename	SHA256
UCBIBanking.sol	ba4c04fa1dfdadd151bc890572769e6ad7ca919bd6b57cbb7d6fe565a346c531

Overview

1. Introduction

This overview documents the role of the UCBI token within the UCBI ecosystem as described in the project whitepaper. It is provided for contextual reference alongside the security audit of the `UCBIBanking` contract and reflects the team's intended approach as communicated through the whitepaper. No part of this overview constitutes an independent verification of the ecosystem or its intended functions.

2. The UCBI Platform

The whitepaper describes the UCBI platform's activities as including:

- The accumulation and management of a digital treasury centred on Ethereum.
- Allocation across additional asset classes, such as real estate.
- The delivery of treasury advisory services.
- Support services for third-party token launches, covering tokenomics design, liquidity structuring, launch strategy advice, preparation for listing, and cash flow sustainability planning.

Within this structure, the whitepaper presents the UCBI token as a functional component of the platform's technological and community ecosystem.

3. Role of the UCBI Token in the Ecosystem

The whitepaper lists the following functional roles for the UCBI token within the platform's ecosystem:

Role	Description
Platform utility	Use of the token in connection with features and services delivered through the UCBI platform.
Community incentives	Use of the token within community-oriented initiatives..
Staking participation	Use of the token in staking arrangements that the whitepaper describes as part of the intended ecosystem design.
Strategic ecosystem expansion	Use of the token in activities that contribute to extending the ecosystem over time.

The whitepaper additionally states that the token enhances liquidity and community engagement, and references governance potential as a direction subject to further review.

The whitepaper further characterises the token as a structural element of the broader UCBI ecosystem. The roles described above reflect the team's intended approach as communicated through the whitepaper.

4. Relationship to the Audited Contract

The `UCBIBanking` contract reviewed in the accompanying audit report provides the on-chain substrate on which the UCBI token operates.

At the contract level, this substrate consists of:

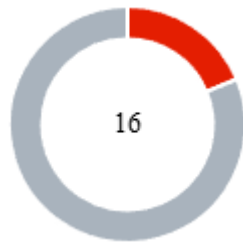
- A transferable ERC20 unit with a fixed total supply of 12,000,000 units and 8 decimals.
- A burn capability that allows holders to permanently destroy their own tokens.
- A centralized transfer-control mechanism that allows the contract owner to pause transfers globally or selectively restrict individual addresses through on-chain allowlist and blocklist mappings.

The ecosystem functions referenced in the whitepaper including platform utility accrual, community-incentive distribution, staking participation, governance arrangements, liquidity-enhancement mechanisms, and other forms of strategic ecosystem expansion — are not implemented within the audited contract. Their delivery would rely on additional smart contracts or off-chain processes that are not part of the reviewed codebase. This reflects the team's intended approach as described in the whitepaper.

5. Scope of this Overview

The content of this overview reflects the team's intended approach as communicated through the whitepaper. It is provided for contextual reference and does not represent an independent verification of the ecosystem functions, the platform's outlook, or the future implementation of any component described in the whitepaper.

Findings Breakdown



● Critical	3
● Medium	0
● Minor / Informative	13

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	0	3	0	0
● Medium	0	0	0	0
● Minor / Informative	0	13	0	0

ST - Stops Transactions

Criticality	Critical
Location	UCBIBanking.sol#L111,123,127
Status	Acknowledged

Description

The contract owner has the authority to stop the transactions for all users excluding the owner and the addresses listed in the `allowedAddresses` mapping. The owner may take advantage of it by setting the `locked` variable to true via the `setLocked` method. The `locked` flag is initialized to true at contract construction, which means transfers are disabled by default until the owner explicitly calls `setLocked(false)`. Additionally, the owner is able to lock the pair or router address which will not allow users to purchase or sell tokens from or to the pair. As a result, the contract may operate as a honeypot.

```
Shell
bool public locked = true;

function setLocked(bool _locked) public onlyOwner {(...)}

function canTransfer(address _addr) public constant returns
(bool) {
    if(locked){
        if(!allowedAddresses[_addr]&&_addr!=owner) return
false;
    }else if(lockedAddresses[_addr]) return false;
    return true;
}
```

Recommendation

The contract could embody a check for not allowing setting the `locked` state to true once trading has been enabled. A suggested implementation could check that the `locked flag` cannot be re-enabled after it has been initially disabled. Additionally, important contract address should always be treated as excluded from conditions that restrict the from transferring tokens.

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions. Temporary Solutions: These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

Team Update

The team has acknowledged that this is not a security issue and states:

In our case, the UCBI_Banking contract is intentionally designed with a degree of centralization for economic, organizational, and regulatory reasons:

Temporary wallet restrictions

Our economic model requires certain lock-up periods. This allows us to:

- stabilize the token market
- prevent excessive short-term speculation
- meet commitments made to our investors

Transfer limitations

Transfers may be restricted because our shareholders and partners are currently operating within a centralized process. This ensures:

- better control over financial flows
- compliance with our internal structure
- a gradual transition before broader openness

Contract owner role

At this stage, we cannot:

- remove the owner role
- nor renounce ownership

because governance is still centralized. As a structured company, this enables us to:

- maintain operational security
- intervene quickly if needed
- manage the project in line with our roadmap

Regulatory compliance (KYC/AML)

Our company operates within a regulatory framework that requires strict compliance with Know Your Customer (KYC) and Anti-Money Laundering (AML) procedures. As a result, we must retain the technical ability to:

- monitor and control transactions when necessary
- restrict or block wallets involved in suspicious activities
- act in accordance with regulations related to anti-money laundering and counter-terrorism financing

These measures are essential to ensure legal compliance, protect the ecosystem, and align with the obligations imposed on us as a regulated entity.

Conclusion

In summary, the mechanisms identified (blacklisting, pausing transactions, centralized control) are not considered vulnerabilities in our context, but rather intentional design choices aligned with our current development phase, economic model, and regulatory obligations.

That said, we remain open to evolving the architecture (e.g., multi-signature mechanisms, progressive decentralization) as our ecosystem and governance mature.

BC - Blacklists Addresses

Criticality	Critical
Location	UCBIBanking.sol#L110,118
Status	Acknowledged

Description

The contract owner has the authority to stop addresses from transactions. The owner may take advantage of it by calling the `lockAddress` function.

```
Shell
mapping(address => bool) public lockedAddresses;
function lockAddress(address _addr, bool _locked)
public onlyOwner {(...)}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

Team Update

The team has acknowledged that this is not a security issue and states:

In our case, the UCBI_Banking contract is intentionally designed with a degree of centralization for economic, organizational, and regulatory reasons:

Temporary wallet restrictions

Our economic model requires certain lock-up periods. This allows us to:

- stabilize the token market
- prevent excessive short-term speculation
- meet commitments made to our investors

Transfer limitations

Transfers may be restricted because our shareholders and partners are currently operating within a centralized process. This ensures:

- better control over financial flows
- compliance with our internal structure
- a gradual transition before broader openness

Contract owner role

At this stage, we cannot:

- remove the owner role
- nor renounce ownership

because governance is still centralized. As a structured company, this enables us to:

- maintain operational security
- intervene quickly if needed
- manage the project in line with our roadmap

Regulatory compliance (KYC/AML)

Our company operates within a regulatory framework that requires strict compliance with Know Your Customer (KYC) and Anti-Money Laundering (AML) procedures. As a result, we must retain the technical ability to:

- monitor and control transactions when necessary
- restrict or block wallets involved in suspicious activities
- act in accordance with regulations related to anti-money laundering and counter-terrorism financing

These measures are essential to ensure legal compliance, protect the ecosystem, and align with the obligations imposed on us as a regulated entity.

Conclusion

In summary, the mechanisms identified (blacklisting, pausing transactions, centralized control) are not considered vulnerabilities in our context, but rather intentional design choices aligned with our current development phase, economic model, and regulatory obligations.

That said, we remain open to evolving the architecture (e.g., multi-signature mechanisms, progressive decentralization) as our ecosystem and governance mature.

ITC - Insufficient Transfer Check

Criticality	Critical
Location	UCBIBanking.sol#L197
Status	Acknowledged

Description

The contract implements transfer restrictions through the `canTransfer` function but only checks the `msg.sender` inside `transferFrom` instead of the actual token holder. As a result, a restricted or locked address may still have its tokens transferred by another approved address, bypassing the intended transfer restrictions. In addition, approvals remain available while transfers are locked, which further enables this behavior by allowing users to pre-approve third parties that can later move their tokens indirectly.

Shell

```
function transferFrom(address _from, address _to,
uint256 _value) public returns (bool) {
    require(_to != address(0));
    require(canTransfer(msg.sender));
    ...
}
```

Recommendation

The team is advised to update the `transferFrom` logic to validate the transfer permissions of the token owner in addition to the caller. This ensures that restricted addresses cannot bypass transfer limitations through previously approved allowances.

Team Update

The team has acknowledged that this is not a security issue and states:

In our case, the UCBI_Banking contract is intentionally designed with a degree of centralization for economic, organizational, and regulatory reasons:

Temporary wallet restrictions

Our economic model requires certain lock-up periods. This allows us to:

- stabilize the token market
- prevent excessive short-term speculation
- meet commitments made to our investors

Transfer limitations

Transfers may be restricted because our shareholders and partners are currently operating within a centralized process. This ensures:

- better control over financial flows
- compliance with our internal structure
- a gradual transition before broader openness

Contract owner role

At this stage, we cannot:

- remove the owner role
- nor renounce ownership

because governance is still centralized. As a structured company, this enables us to:

- maintain operational security
- intervene quickly if needed
- manage the project in line with our roadmap

Regulatory compliance (KYC/AML)

Our company operates within a regulatory framework that requires strict compliance with Know Your Customer (KYC) and Anti-Money Laundering (AML) procedures. As a result, we must retain the technical ability to:

- monitor and control transactions when necessary
- restrict or block wallets involved in suspicious activities
- act in accordance with regulations related to anti-money laundering and counter-terrorism financing

These measures are essential to ensure legal compliance, protect the ecosystem, and align with the obligations imposed on us as a regulated entity.

Conclusion

In summary, the mechanisms identified (blacklisting, pausing transactions, centralized control) are not considered vulnerabilities in our context, but rather intentional design choices aligned with our current development phase, economic model, and regulatory obligations.

That said, we remain open to evolving the architecture (e.g., multi-signature mechanisms, progressive decentralization) as our ecosystem and governance mature.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	UCBIBanking.sol#L79,113,118,123
Status	Acknowledged

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

Shell

```
function setLocked(bool _locked) public onlyOwner
{(...)}
function allowAddress(address _addr, bool _allowed)
public onlyOwner {(...)}
function lockAddress(address _addr, bool _locked)
public onlyOwner {(...)}

function transferOwnership(address newOwner)
onlyOwner public {(...)}
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

Team Update

The team has acknowledged that this is not a security issue and states:

In our case, the UCBI_Banking contract is intentionally designed with a degree of centralization for economic, organizational, and regulatory reasons:

Temporary wallet restrictions

Our economic model requires certain lock-up periods. This allows us to:

- stabilize the token market
- prevent excessive short-term speculation
- meet commitments made to our investors

Transfer limitations

Transfers may be restricted because our shareholders and partners are currently operating within a centralized process. This ensures:

- better control over financial flows
- compliance with our internal structure
- a gradual transition before broader openness

Contract owner role

At this stage, we cannot:

- remove the owner role
- nor renounce ownership

because governance is still centralized. As a structured company, this enables us to:

- maintain operational security
- intervene quickly if needed
- manage the project in line with our roadmap

Regulatory compliance (KYC/AML)

Our company operates within a regulatory framework that requires strict compliance with Know Your Customer (KYC) and Anti-Money Laundering (AML) procedures. As a result, we must retain the technical ability to:

- monitor and control transactions when necessary
- restrict or block wallets involved in suspicious activities
- act in accordance with regulations related to anti-money laundering and counter-terrorism financing

These measures are essential to ensure legal compliance, protect the ecosystem, and align with the obligations imposed on us as a regulated entity.

Conclusion

In summary, the mechanisms identified (blacklisting, pausing transactions, centralized control) are not considered vulnerabilities in our context, but rather intentional design choices aligned with our current development phase, economic model, and regulatory obligations.

That said, we remain open to evolving the architecture (e.g., multi-signature mechanisms, progressive decentralization) as our ecosystem and governance mature.

IAU - Inefficient Assert Usage

Criticality	Minor / Informative
Location	UCBIBanking.sol#L18
Status	Acknowledged

Description

The contract uses a `SafeMath` library where arithmetic checks rely on `assert` instead of `require` for handling invalid conditions such as overflows and underflows. As a result, when a failure occurs, the transaction consumes all remaining gas instead of reverting gracefully, which leads to inefficient error handling and unnecessary gas loss for users.

Shell

```
assert(a == 0 || c / a == b);  
assert(b <= a);  
assert(c >= a);
```

Recommendation

The team is advised to replace `assert` statements with `require` for input validation and arithmetic checks to ensure that transactions fail in a controlled and gas-efficient manner. Alternatively, upgrading to a newer Solidity version with built-in overflow and underflow protection is recommended to follow modern best practices.

Team Update

The team has acknowledged that this is not a security issue.

MC - Missing Check

Criticality	Minor / Informative
Location	UCBIBanking.sol#L25,149,204,283
Status	Acknowledged

Description

The contract is processing variables that have not been properly sanitized and checked that they form the proper shape. These variables may produce vulnerability issues.

Specifically the contract does not ensure that the functions revert early on any instant an incorrect subtraction or division happen with the appropriate error message.

```
Shell
uint256 c = a / b;
...
balances[msg.sender] =
balances[msg.sender].sub(_value);
...
balances[_from] = balances[_from].sub(_value);
...
allowed[_from][msg.sender] = _allowance.sub(_value);
...
balances[burner] = balances[burner].sub(_value);
```

Recommendation

The team is advised to properly check the variables according to the required specifications.

Team Update

The team has acknowledged that this is not a security issue

MEM - Missing Error Messages

Criticality	Minor / Informative
Location	UCBIBanking.sol#L70,80,114,119,144,145,196,197,277,278
Status	Acknowledged

Description

The contract is missing error messages. Specifically, there are no error messages to accurately reflect the problem, making it difficult to identify and fix the issue. As a result, the users will not be able to find the root cause of the error.

```
Shell
require(msg.sender == owner)
require(newOwner != address(0))
require(_addr != owner)
require(_to != address(0))
require(canTransfer(msg.sender))
require(_value > 0)

require(_value <= balances[msg.sender])
```

Recommendation

The team is suggested to provide a descriptive message to the errors. This message can be used to provide additional context about the error that occurred or to explain why the contract execution was halted. This can be useful for debugging and for providing more information to users that interact with the contract.

Team Update

The team has acknowledged that this is not a security issue.

MEE - Missing Events Emission

Criticality	Minor / Informative
Location	UCBIBanking.sol#L113,118,123
Status	Acknowledged

Description

The contract performs actions and state mutations from external methods that do not result in the emission of events. Emitting events for significant actions is important as it allows external parties, such as wallets or dApps, to track and monitor the activity on the contract. Without these events, it may be difficult for external parties to accurately determine the current state of the contract.

Shell

```
function allowAddress(address _addr, bool _allowed)
public onlyOwner {...}

function lockAddress(address _addr, bool _locked)
public onlyOwner {...}

function setLocked(bool _locked) public onlyOwner
{...}
```

Recommendation

It is recommended to include events in the code that are triggered each time a significant action is taking place within the contract. These events should include relevant details such as the user's address and the nature of the action taken. By doing so, the contract will be more transparent and easily auditable by external parties. It will also help prevent potential issues or disputes that may arise in the future.

Team Update

The team has acknowledged that this is not a security issue.

MRO - Missing Renounce Ownership

Criticality	Minor / Informative
Location	UCBIBanking.sol#L50
Status	Acknowledged

Description

The contract inherits ownership functionality through Ownable but does not provide a way for the owner to renounce ownership and remove privileged control. As a result, the owner will always retain full administrative authority over sensitive functions such as locking transfers and managing address permissions, which introduces long-term centralization and trust concerns.

```
Shell
contract Ownable {...}
```

Recommendation

The team is advised to implement a renounce ownership mechanism that allows the owner to permanently remove their privileges once they are no longer needed. This helps increase trust in the system by ensuring that administrative control can be fully relinquished.

Team Update

The team has acknowledged that this is not a security issue.

MTEE - Missing Transfer Event Emission

Criticality	Minor / Informative
Location	UCBIBanking.sol#L301
Status	Acknowledged

Description

The contract does not emit an event when portions of the main amount are transferred during the transfer process. This lack of event emission results in decreased transparency and traceability regarding the flow of tokens, and hinders the ability of decentralized applications (dApps), such as blockchain explorers, to accurately track and analyze these transactions.

Shell

```
function UCBIBanking () {  
    totalSupply = initialSupply;  
    balances[msg.sender] = initialSupply;  
    allowedAddresses[owner] = true;  
}
```

Recommendation

It is advisable to incorporate the emission of detailed event logs following each asset transfer. These logs should encapsulate key transaction details, including the identities of the sender and receiver, and the quantity of assets transferred. Implementing this practice will enhance the reliability and transparency of transaction tracking systems, ensuring accurate data availability for ecosystem participants.

Team Update

The team has acknowledged that this is not a security issue.

NDAB - Non-Reverting Decrease Approval Behavior

Criticality	Minor / Informative
Location	UCBIBanking.sol#L253
Status	Acknowledged

Description

The `decreaseApproval` function reduces the allowance but sets it to zero when the requested reduction exceeds the current allowance instead of reverting. As a result, this silent adjustment may lead to unexpected behavior and reduce clarity for users and integrators who expect strict validation.

Shell

```
function decreaseApproval (address _spender, uint
_subtractedValue)
    returns (bool success) {
        uint oldValue = allowed[msg.sender][_spender];
        if (_subtractedValue > oldValue) {
            allowed[msg.sender][_spender] = 0;
        }
        ...
    }
```

Recommendation

The team is advised to enforce stricter checks by reverting the transaction when the reduction amount exceeds the current allowance. This improves predictability and aligns the function behavior with common expectations for safe allowance management.

Team Update

The team has acknowledged that this is not a security issue.

NWES - Nonconformity with ERC-20 Standard

Criticality	Minor / Informative
Location	UCBIBanking.sol#L92
Status	Acknowledged

Description

The contract does not fully conform to the ERC20 standard. Specifically, the standard defines `totalSupply` as a function that should be explicitly exposed to return the total number of tokens in existence, whereas the contract only declares it as a public state variable without providing a dedicated function implementation. As a result, this deviation from the standard may lead to inconsistencies or compatibility issues with external integrations and tools that expect a strict ERC20 interface.

```
Shell
uint256 public totalSupply;
```

Recommendation

The incorrect implementation of the ERC20 standard could potentially lead to problems when interacting with the contract, as other contracts or applications that expect the ERC20 interface may not behave as expected. The team is advised to review and revise the implementation of the transfer mechanism to ensure full compliance with the ERC20 standard. <https://eips.ethereum.org/EIPS/eip-20>.

Team Update

The team has acknowledged that this is not a security issue.

OCV - Outdated Compiler Version

Criticality	Minor / Informative
Location	UCBIBanking.sol#L1
Status	Acknowledged

Description

The contract is compiled with Solidity `^0.4.23`, a significantly outdated release of the Solidity compiler. Older compiler versions, particularly those in the 0.4.x series, contain known bugs and lack modern security features that have been introduced in later versions.

```
Shell  
pragma solidity ^0.4.23;
```

Recommendation

The team is advised to migrate the contract to a modern Solidity compiler version.

Team Update

The team has acknowledged that this is not a security issue.

L04 - Conformance to Solidity Naming Conventions

Criticality	Minor / Informative
Location	UCBIBanking.sol#L113,118,123,127,143,160,195,221,233,243,250,276
Status	Acknowledged

Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX_VALUE, ERROR_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

Shell

```
bool _allowed
address _addr
bool _locked
uint256 _value
address _to
address _owner
address _from
address _spender
```

```
uint _addedValue  
uint _subtractedValue
```

Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.

Find more information on the Solidity documentation

<https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions>.

Team Update

The team has acknowledged that this is not a security issue.

L09 - Dead Code Elimination

Criticality	Minor / Informative
Location	UCBIBanking.sol#L19,25
Status	Acknowledged

Description

In Solidity, dead code is code that is written in the contract, but is never executed or reached during normal contract execution. Dead code can occur for a variety of reasons, such as:

- Conditional statements that are always false.
- Functions that are never called.
- Unreachable code (e.g., code that follows a return statement).

Dead code can make a contract more difficult to understand and maintain, and can also increase the size of the contract and the cost of deploying and interacting with it.

Shell

```
function mul(uint256 a, uint256 b) internal constant  
returns (uint256) {...}
```

```
function div(uint256 a, uint256 b) internal constant  
returns (uint256) {...}
```

Recommendation

To avoid creating dead code, it's important to carefully consider the logic and flow of the contract and to remove any code that is not needed or that is never executed. This can help improve the clarity and efficiency of the contract.

Team Update

The team has acknowledged that this is not a security issue.

L19 - Stable Compiler Version

Criticality	Minor / Informative
Location	UCBIBanking.sol#L1
Status	Acknowledged

Description

The `^` symbol indicates that any version of Solidity that is compatible with the specified version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

Shell

```
pragma solidity ^0.4.23;
```

Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

Team Update

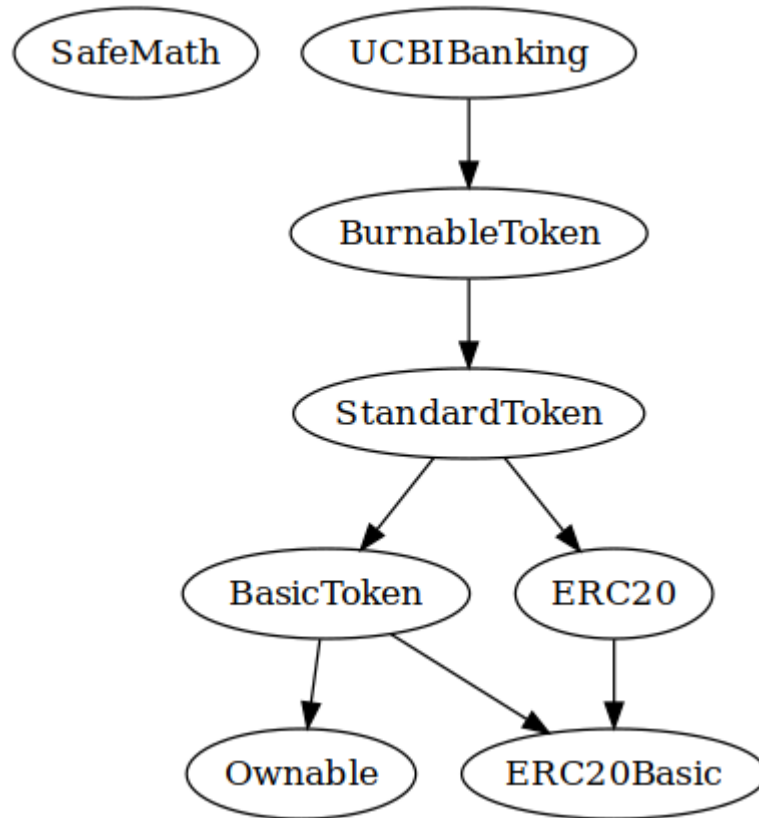
The team has acknowledged that this is not a security issue.

Functions Analysis

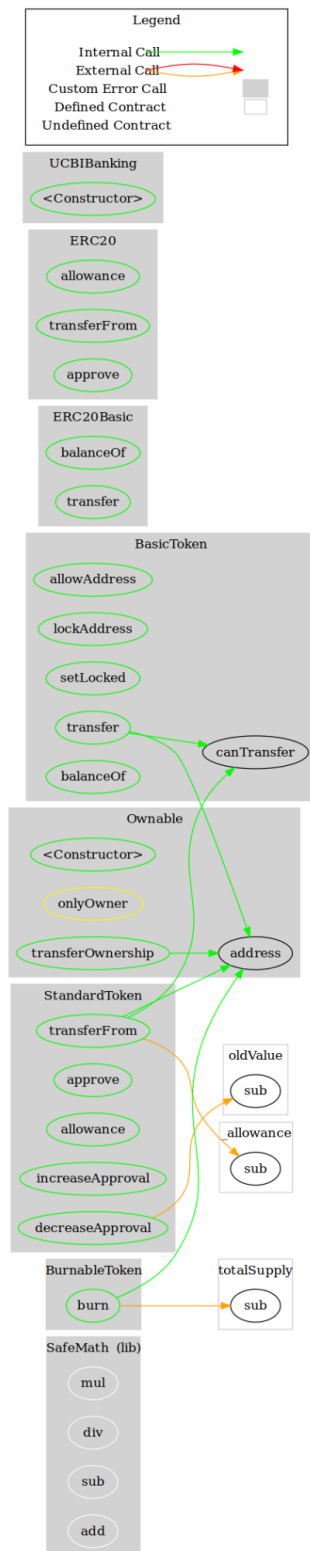
Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
ERC20Basic	Implementation			
	balanceOf	Public		-
	transfer	Public	✓	-
BasicToken	Implementation	ERC20Basic, Ownable		
	allowAddress	Public	✓	onlyOwner
	lockAddress	Public	✓	onlyOwner
	setLocked	Public	✓	onlyOwner
	canTransfer	Public		-
	transfer	Public	✓	-
	balanceOf	Public		-
ERC20	Implementation	ERC20Basic		
	allowance	Public		-
	transferFrom	Public	✓	-
	approve	Public	✓	-
StandardToken	Implementation	ERC20, BasicToken		

	transferFrom	Public	✓	-
	approve	Public	✓	-
	allowance	Public		-
	increaseApproval	Public	✓	-
	decreaseApproval	Public	✓	-
BurnableToken	Implementation	StandardToken		
	burn	Public	✓	-
UCBIBanking	Implementation	BurnableToken		
		Public	✓	-

Inheritance Graph



Flow Graph



Summary

UCBI_Banking contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements. There are some functions that can be abused by the owner like stop transactions and massively blacklist addresses. A multi-wallet signing pattern will provide security against potential hacks. Temporarily locking the contract or renouncing ownership will eliminate all the contract threats.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io